数字逻辑基础与Verilog设计
---时序篇

本章内容

- 锁存器
- 边沿触发的D触发器
- T和JK触发器
  - D触发器构成T、JK触发器
  - T，JK触发器的综合问题
- 寄存器
- 计数器

3.1 D触发器应用：构建T触发器

T触发器（Toggle Flip－Flop）：

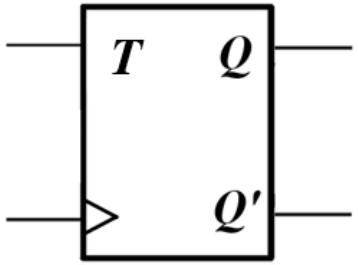
- 功能：具有保持和翻转功能的触发器。
- 特性表：

Clk	T	Q(t+1)
0/1	×	Q(t)
↑	0	Q(t)
↑	1	Q'(t)

$$Q(t+1) = Q(t) \oplus T$$

采用D触发器构建T触发器：

$$D = Q(t) \oplus T$$



图形符号

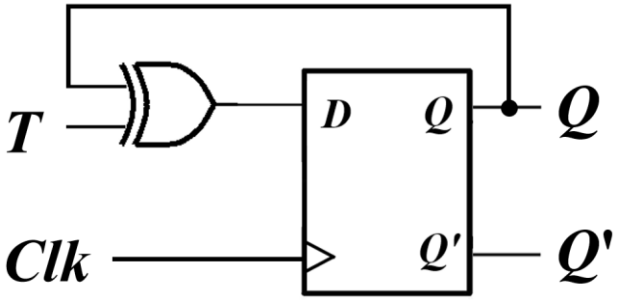
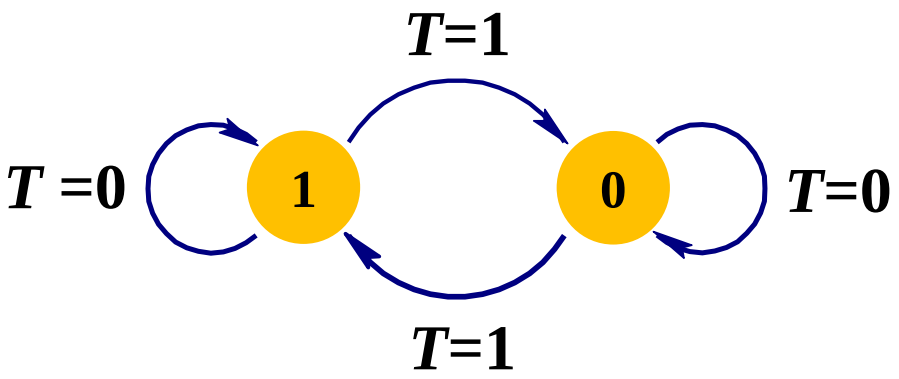


图5.15 D触发器实现T触发器电路

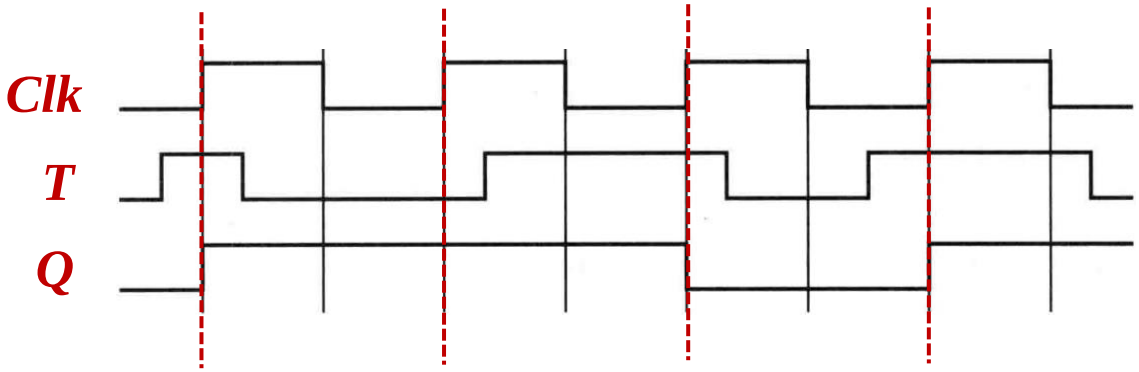
3.1 D触发器应用：构建T触发器

T触发器（Toggle Flip－Flop）：

- 状态转移图：



- 时序图：



3.1 D触发器应用：构建JK触发器

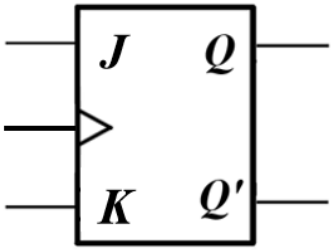
JK触发器（JK Flip－Flop）：

- 功能：具有置0、置1、保持和翻转功能的触发器。
- 特性表：

J	K	$Q(t+1)$
0	0	$Q(t)$
0	1	0
1	0	1
1	1	$Q'(t)$

$$Q(t+1) = JQ'(t) + K'Q(t)$$

令：$D = JQ'(t) + K'Q(t)$



图形符号

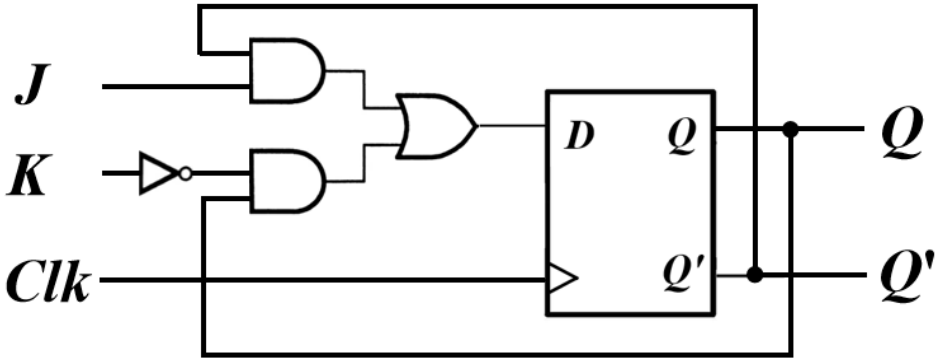
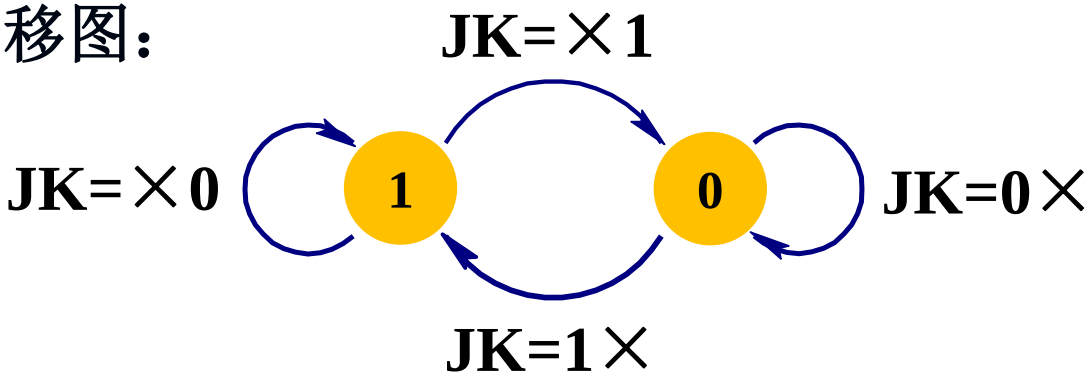


图5.16 上升沿触发的JK触发器

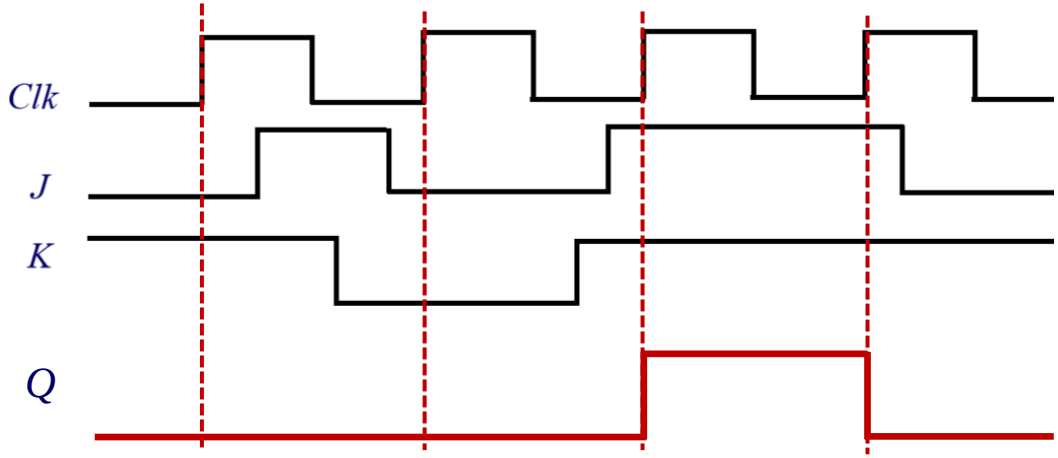
3.1 D触发器应用：构建JK触发器

JK触发器（JK Flip－Flop）：

- 状态转移图：



- 时序图：



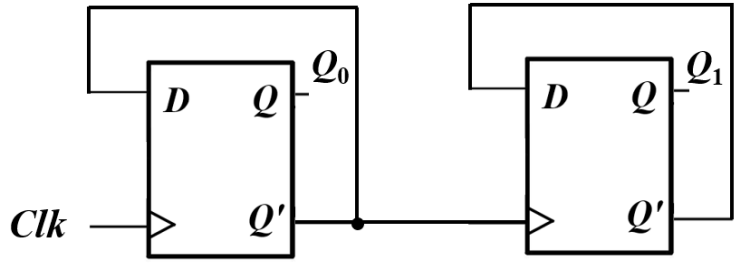
JK=00：保持

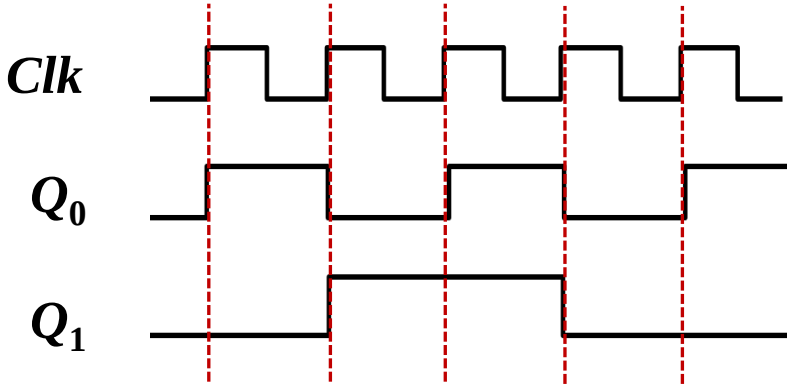
JK=11：翻转

J≠K，$Q(t+1)=J$

3.1 D触发器应用：作分频器

分频电路：

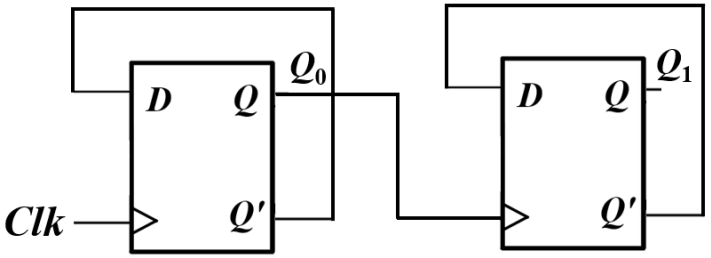




Q_0为*Clk*的二分频；

Q_1为*Clk*的四分频；

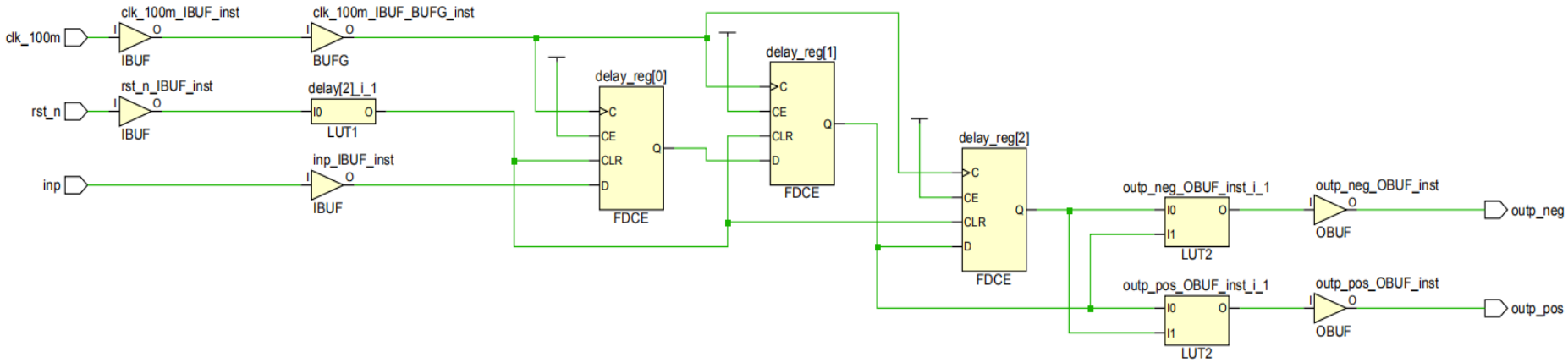
思考：如果将Q_0接到右边*Clk*，结果如何？



3.1 D触发器应用：脉冲边沿检测

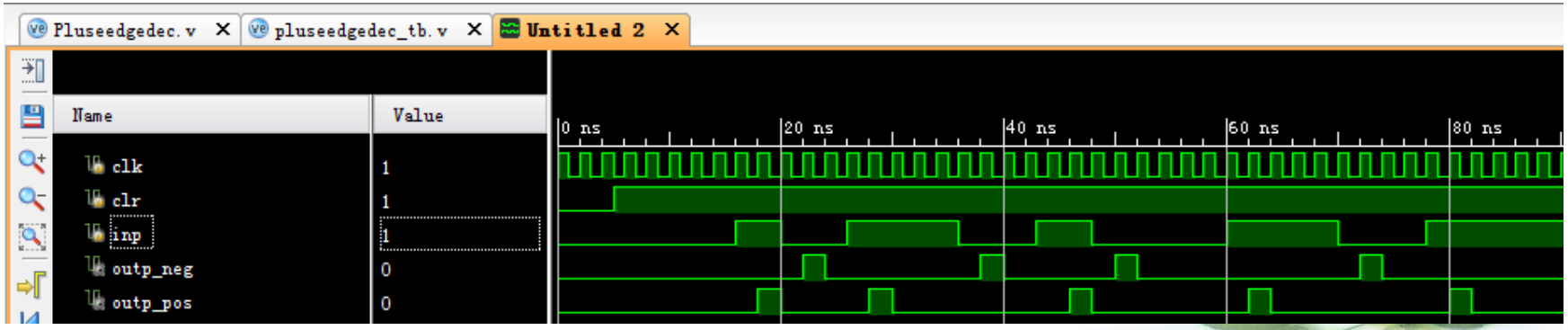
2.脉冲边沿检测： 在众多应用中需要检测某一个脉冲信号的下降沿或上升沿。

```
module Pluseedgedec(
    input clk,rst_n, inp,
    output outp_neg, outp_pos
);
    reg [2:0] delay;
    always @(posedge clk or negedge rst_n) begin
        if(!rst_n)   delay <= 3'b000;
        else         delay <={delay[1:0],inp};
    end
    assign  outp_neg = delay[2] & ~delay[1];
    assign  outp_pos = delay[1] & ~delay[2];
endmodule
```



3.1 D触发器应用：脉冲边沿检测

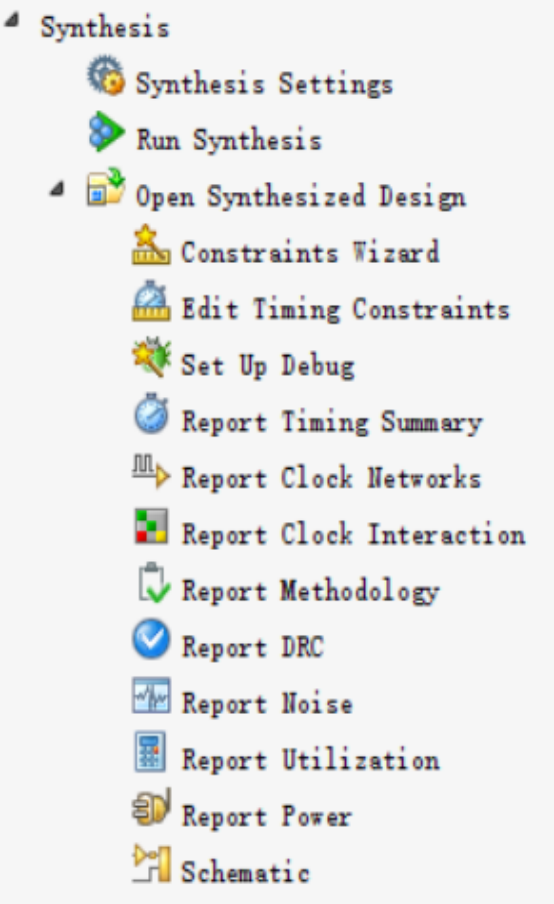
```
//Testbench测试代码
module pluseedgedec_tb();
    reg clk;
    reg rst_n;
    reg inp;
    wire outp_neg;
    wire outp_pos;
    Pluseedgedec U1(
        .clk(clk),
        .rst_n(rst_n),
        .inp(inp),
        .outp_neg(outp_neg),
        .outp_pos(outp_pos) );

initial begin
    inp = 0;
    #16 inp =1;
    #4 inp = 0;
    #6 inp = 1;
    #10 inp =0;
    #7 inp = 1;
    #5 inp = 0;
    #12 inp =1;
    #10 inp =0;
    #8 inp =1;
end

initial begin
    clk = 1;
    rst_n = 0;
    #5 rst_n = 1;
end
always #1 clk=~clk;
endmodule
```



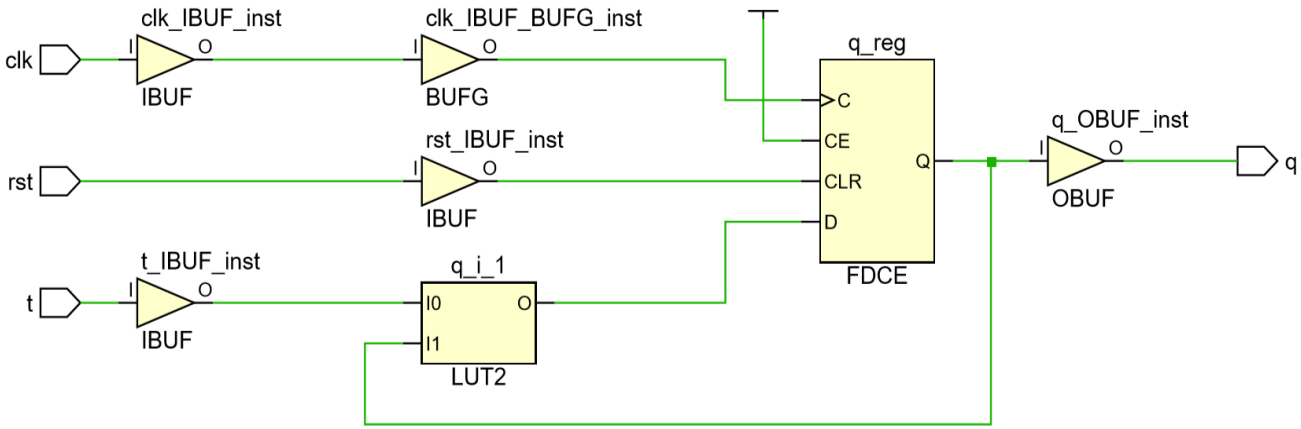
3.2 T触发器的综合问题

T触发器综合代码：

```
module t_ff(
    input clk, rst, t,
    output reg q
);
always@(posedge clk, posedge rst)
  if(rst)
        q<=1'b0;
  else if (t==1'b1)
        q <= ~q;
  else
        q <= q;
endmodule
```

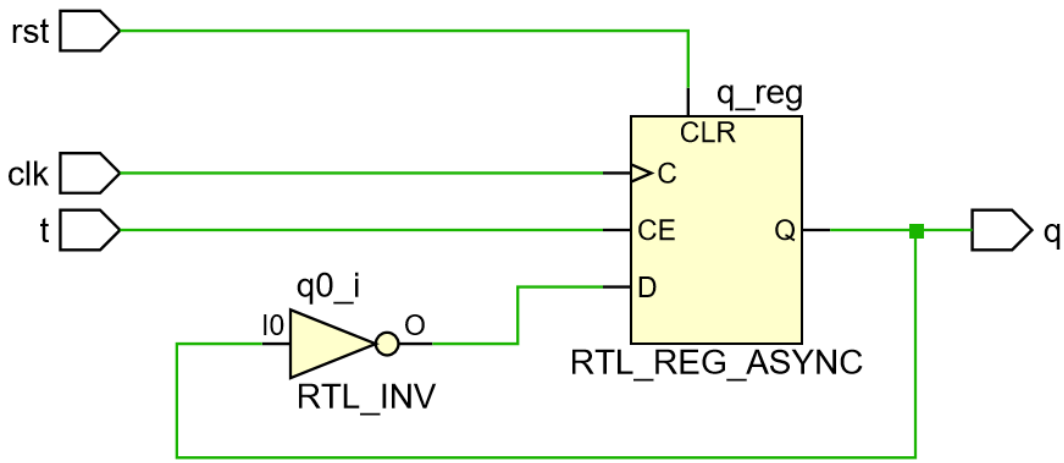


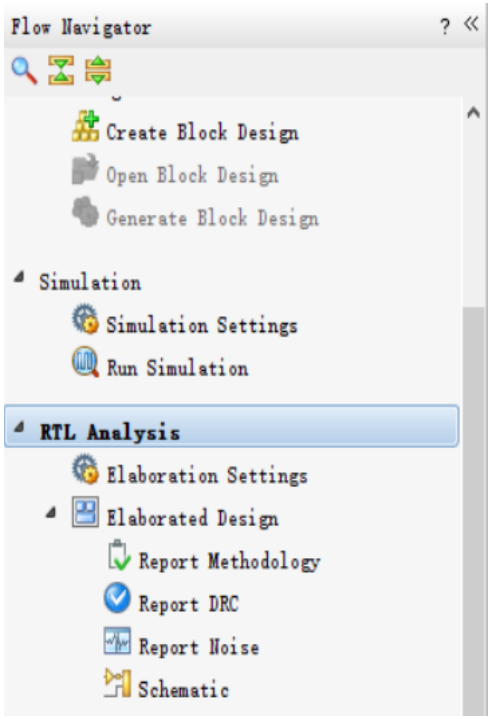
3.2 T触发器的综合问题



- Technology Schematic由全局时钟缓冲器（BUFGP）、各种输入/输出缓冲（ibuf）（obuf）、LUT（查找表）及D触发器等组成，这些都是FPGA底层的元件。**FPGA中只有D触发器，T触发器是由D触发器综合而成的。**说明**Technology Schematic是代码在FPGA中综合的最终结果**。反映了实际的电路和资源使用情况。因此，Technology Schematic是FPGA实际的电路图，是基于所使用器件的结构的。

3.2 T触发器的综合问题





- RTL Schematic是语法分析得到的结果，类似于用原理图设计电路，是设计输入最忠实的体现，并非设计的最终电路结构。主要作用是帮助设计者理解设计的算法、检查设计输入中的问题。RTL Viewer与目标器件是独立的。初学者可以将源码和RTL原理图对照学习。

3.2 T触发器的综合问题

关于FPGA中的查找表LUT

•FPGA是在PAL、GAL、CPLD等可编程器件的基础上进一步发展的产物。它是一种半定制电路，既解决了定制电路的不足，又克服了原有可编程器件门电路有限的缺点。FPGA需要被反复烧写，它采用一种易于反复配置的结构来实现组合逻辑的基本结构。

•目前主流FPGA均采用了基于SRAM工艺的查找表结构。通过烧写文件改变查找表内容的方法来实现对FPGA的重复配置。对于一个n输入的逻辑运算，最多只可能存在2^n种结果。所以如果事先将相应的结果存放于一个存贮单元，就相当于实现了门电路的功能。

3.2 T触发器的综合问题

关于FPGA中的查找表LUT（接1）

•查找表（Look-Up-Table）简称为LUT，LUT本质上就是一个RAM。目前FPGA中多使用4输入的LUT，每一个LUT可以看成一个有4位地址线的RAM。 当用户通过原理图或HDL语言描述了一个逻辑电路后，FPGA开发软件会自动计算逻辑电路的所有可能结果，并把真值表（即结果）事先写入RAM，这样，每输入一个信号进行逻辑运算就等于输入一个地址进行查表，找出地址对应的内容输出即可。

3.2 T触发器的综合问题

关于FPGA中的查找表LUT（接2）

- 下面给出一个4输入与门的例子来说明LUT实现逻辑功能的原理。同时给出使用LUT实现4输入与门电路的真值表。LUT具有与逻辑电路相同的功能，但具有更快的执行速度。

实际逻辑电路		LUT实现方式	
a，b，c，d的输入	逻辑输出	RAM地址	RAM中存储的内容
0000	0	0000	0
0001	0	0001	0
…	…	…	…
1111	1	1111	1

3.2 T触发器的综合问题

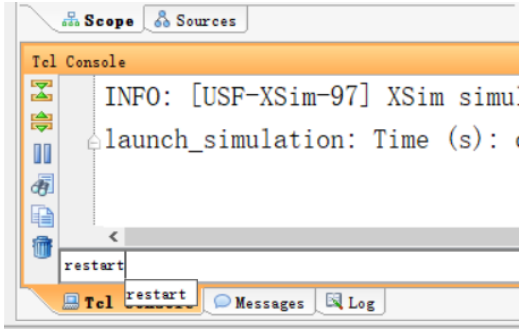
T触发器Testbeach代码

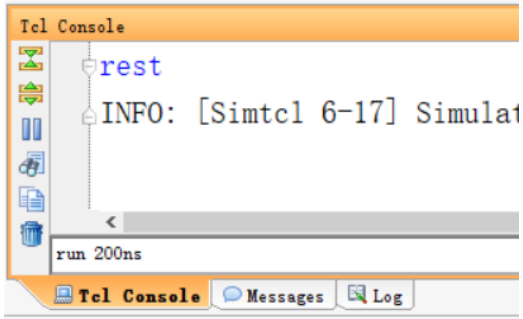
```
module t_ff_tb;
   reg t,clk,rst;  //输入一定是reg型
   wire q;         //输出一定是wire型
   always          //always中放时钟，一定要在initial中初始化
      #10 clk = ~clk;
   t_ff uut1(      //t_ff 实例化
       .clk (clk) ,
       .rst (rst) ,
       .t (t) ,
       .q (q)
   );
```

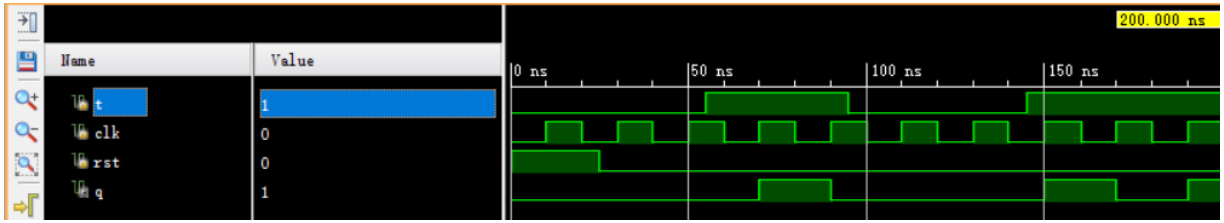
3.2 T触发器的综合问题

T触发器Testbeach代码（接1）

```
initial          // 时钟以外的输入信号在initial中重置
  begin          // initial 过程化中的begin end表示串行运行
  clk=1'b0;
  rst = 1'b1;
  t = 1'b0;
  # 25 rst = 1'b0;
  # 30 t = 1'b1;
  # 40 t = 1'b0;
  # 50 t = 1'b1;
  end
endmodule
```





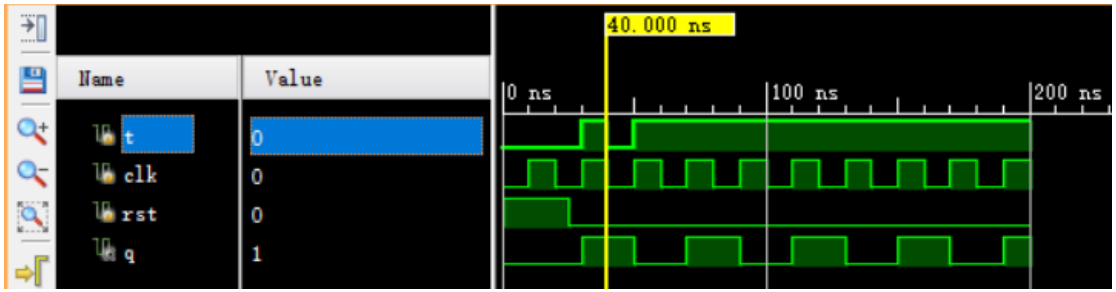


3.2 T触发器的综合问题

T触发器Testbeach代码（接2）

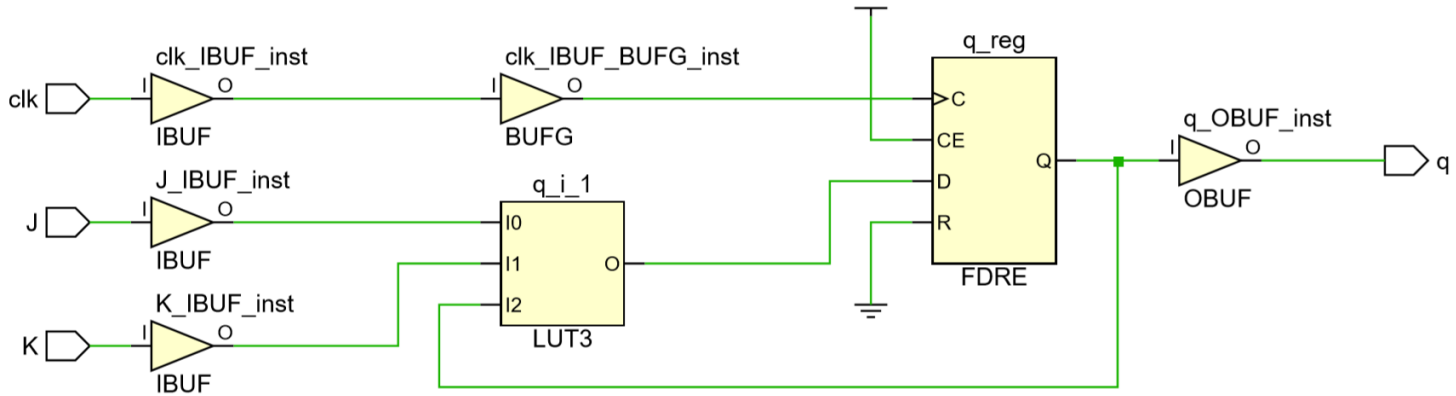
若仿真代码中，若initial块begin end改成fork join：

```
  initial
    fork        // initial 过程化中的 fork join表示并行运行
    clk=1'b0;
    rst = 1'b1;
    t = 1'b0;
    # 25 rst = 1'b0;
    # 30 t = 1'b1;
    # 40 t = 1'b0;
    # 50 t = 1'b1;
    join
endmodule
```



3.3 JK触发器的综合

```
module JK_ff(
   input clk,rst,J, K,
   output reg q
   );
always@(posedge clk, posedge rst)
 if(rst)
       q<=1'b0;
 else
 case({J,K})
        2'b00: q <= q;
        2'b01: q <= 0;
        2'b10: q <= 1;
        2'b11: q <= ~q;
      default: q <= q;
 endcase
endmodule
```



JK_ff ’ Technology Schematic

3.4 分频器的实现

- 分频器是FPGA中常用的设计之一，在FPGA设计中担任重要的角色。尽管大多数设计中会采用厂家集成的锁相环PLL资源进行分频、倍频（每个厂商Xilinx /Alter 等其开发套件会提供各自的IP），但对对时钟要求不高的基本设计还是需要通过自行设计分频，可节省锁相环资源。
- 实验要求：对时钟信号进行偶分频/奇分频，占空比为1:1.

设计思路：

1. 偶数分频

- 假设为N分频，计数到N/2-1时，时钟翻转、计数清零，如此循环就可以得到N（偶）分频。

3.4 分频器的实现

```
module FD_even(input clk_in, rst_n, output reg clk_out );  //偶分频的代码
parameter  N=6;
reg[3:0] cnt;
always@(posedge clk_in or negedge rst_n)
begin
if(!rst_n)
     cnt<= 4’b0000;
else if(cnt == (N/2-1))
     cnt <= 4’b0000;
else
     cnt <= cnt + 1;
end
```

```
always@(posedge clk_in，negedge rst_n)
begin
  if(!rst_n)
        clk_out <= 1’b0;
  else if(cnt == (N/2-1))
        clk_out <= ~ clk_out;
  else clk_out <= clk_out;
end
endmodule
```

3.4 分频器的实现

2.奇数N分频（占空比为50%）

方法1：

- **上升沿触发的模N计数cnt_p，当计数为（N-1）/2-1和N-1时将clk进行翻转，得到一个占空比0.4的N分频clk_p。**
- **下降沿触发的模N计数cnt_n，当计数到（N-1）/2-1和N-1时，将clk进行翻转，得到一个占空比0.4的N分频clk_n。**
- **两个时钟进行相“或”运算，得到占空比为50%的奇数N分频时钟。**

3.4 分频器的实现

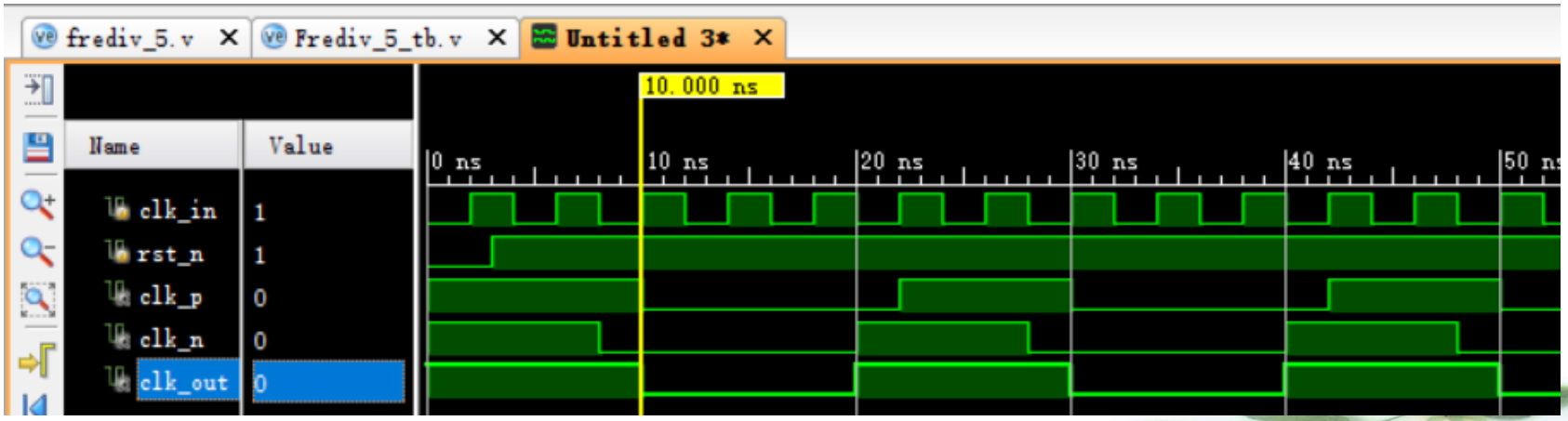
```
module FreDiv_odd(input clk_in, rst_n, output clk_out );
reg [3:0] cnt_p, cnt_n;
reg clk_p,clk_n;
parameter N=5;
assign clk_out = clk_n | clk_p;
always@(posedge clk_in or negedge rst_n)
if(!rst_n)   cnt_p <= 0;
else if(cnt_p== N-1)
            cnt_p<= 0;
      else
            cnt_p<= cnt_p + 1'b1;
```

3.4 分频器的实现

```
always@(posedge clk_in or negedge rst_n)  //上升沿触发
if(!rst_n)   clk_p <= 1; //初值为1，则“或”，为0则“与”
else if(cnt_p== (N-1)/2-1)
            clk_p<= ~clk_p;
     else if(cnt_p== (N-1) )
            clk_p<= ~clk_p;
always@(negedge clk_in or negedge rst_n) //下降沿触发
if(!rst_n)  cnt_n <= 4’d0;
else if(cnt_n== (N-1))
            cnt_n<= 0;
      else
            cnt_n<= cnt_n + 1;
```

3.4 分频器的实现

```
always@(negedge clk_in or negedge rst_n)
if(!rst_n)   clk_n <= 1;
else if(cnt_n== (N-1)/2-1)
            clk_n<= ~clk_n;
      else if(cnt_n== (N-1) )
            clk_n<= ~clk_n;
endmodule
```



3.4 分频器的实现

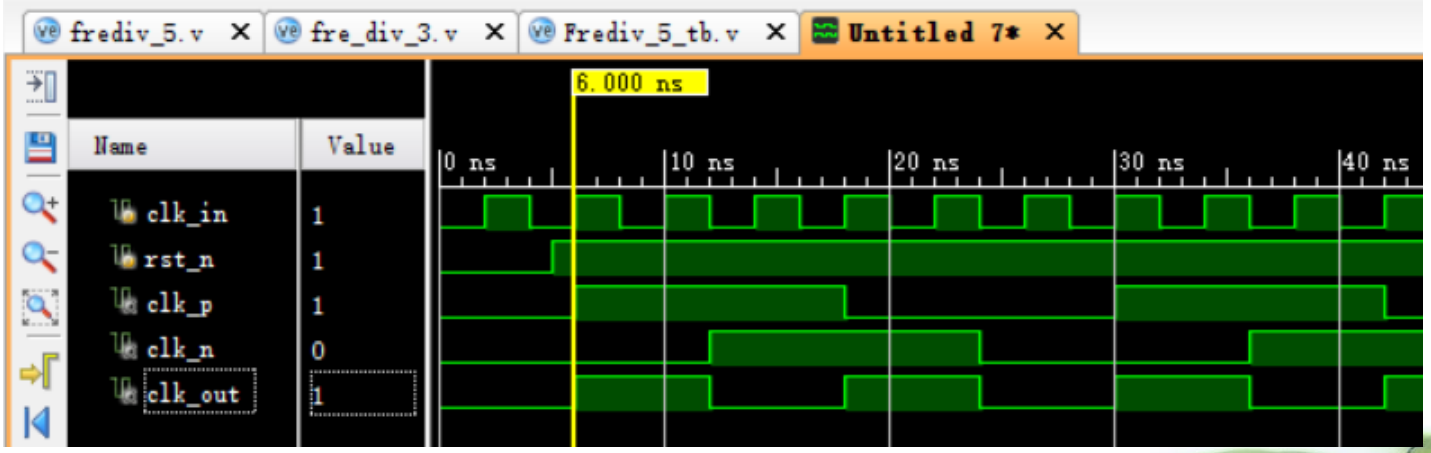
- 方法2：采用上升沿生成计数器N，↑沿在（N-1）/2-1时翻转，得到clk_p；↓沿在N-1时翻转，得到clk_n，二者异或。

```
module fre_div_3(
  input clk_in, rst_n,
  output clk_p,clk_n,
  output clk_out
);
reg[3:0] cnt;
reg clk_p,clk_n;
parameter N=3;
```

```
always@(posedge clk_in or negedge rst_n)
if(!rst_n)  cnt <= 0;
else if(cnt== N-1)
            cnt <= 0;
else        cnt <= cnt + 1'b1;

always@(posedge clk_in or negedge rst_n)

if(!rst_n)   clk_p <= 0;
else if(cnt== (N-1)/2-1)
            clk_p <= ~clk_p;
else        clk_p <= clk_p;
```

3.4 分频器的实现

```
always@(negedge clk_in or negedge rst_n) //下降沿触发
    if(!rst_n)   clk_n <= 0;
    else if(cnt== (N-1))
                 clk_n <= ~clk_n;
    else         clk_n <= clk_n;
assign  clk_out = clk_n ^ clk_p;
endmodule
```



习题：

5.4, 5.6, 5.7, 5.9, 5.10, 5.12

5.17, 5.20, 5.21, 5.22, 5.23, 5.25, 5.28, 5.31